Indicode Compiler Front-End and LLVM IR Status

Status date: 2026-11-04 Package version: 1.0.0 Grammar version: 1.0.0 Milestone: Compiler front end plus textual LLVM IR generation; no machine-code backend.

Executive summary

The requested milestone is complete within the documented single-file front-end v1 and public-API scope:

Area	Completion	Evidence
Front-end v1/API feature catalogue	100% (40/40 items)	Implementation and focused tests at each item's documented acceptance level; IDs 445–484
Textual LLVM IR lowering	100% of documented single-file v1 constructs	IR generator tests and optional LLVM assembly/verifier tests
Combined front-end + textual IR milestone	100% of the defined scope	The supported source subset lowers through verified HIR to textual LLVM IR
Current test run	28/28 passed	26 base CTest cases and two optional tests using already-installed `llvm-as` and `opt`

This is a scoped feature-completion statement, not a claim that every conceivable Indicode feature or a production-ready compiler is complete. The front end accepts the documented single-file v1 grammar. A few separately implemented resolver/index library APIs are not connected to that grammar or an LSP. The exact accepted syntax is in LANGUAGE_SPEC_V1.md; static rules are in SEMANTICS_V1.md.

What “100%” covers

The front end reads source, tokenizes and parses supported constructs, checks names/types/semantics, produces typed HIR, and independently verifies that HIR. The IR generator then lowers all documented single-file v1 constructs from verified HIR to textual LLVM IR.

Front-end area	Delivered v1/API scope
Source and diagnostics	Source buffers/spans, line and column mapping, UTF-8 validation, source-aware diagnostics, fix-its, and diagnostic JSON
Lexer and parser	Documented token/literal forms, recursive-descent declarations/statements, Pratt expressions, recovery, grammar version 1.0.0
AST	Owned AST, deterministic printer, visitor, JSON schema v2, documented ownership/lifetime
Names and semantics	Lexical scopes, stable symbols, name/type checks, generic `Numeric`/`Integer` inference, effects, limited ownership checks, constant and control-flow checks
Typed HIR	AST lowering, schema-versioned output, independent validation of supported types, scopes, calls, effects, ownership metadata, and returns
Supporting APIs	Module/import/overload resolution, HIR symbol/reference index, and incremental parsing implemented and tested at their stated library-API scope
CLI and packaging	`check`, `tokens`, `ast`, `ast-json`, `typed`, `typed-json`, `ir`, and CMake install/export package

The complete 40-item audit, including evidence and per-item boundaries, is maintained in FRONT_END_REMAINING_WORK.md. “Remaining work” in that audit means optional future integration or broader acceptance beyond this milestone, not an unfinished item in the 40-item catalogue at its stated v1/API scope.

LLVM IR lowering coverage

Construct family	Lowering status
Functions and calls	Non-generic functions; on-demand generic specialization, including nested and recursive generic calls
Values and data	Parameters, locals, globals, assignment, records, lists, ranges, text, and opaque-pointer tensor values
Expressions	Integer/floating arithmetic, comparisons, unary operators, checked-conversion helper calls, ownership wrappers, short-circuit Boolean logic
Control flow	`if`/`else if`/`else`, `while`, `repeat`, and `for` over integer ranges, lists, or text
Program initialization	Generated `indicode_entry` initializer when top-level statements exist
Output	Valid textual LLVM IR for supported constructs; failures produce diagnostics rather than partial IR

The detailed lowering contract and type mappings are in LLVM_IR_GENERATOR_V0.md.

Explicitly outside this milestone

These are intentional boundaries, not claims of incomplete work on the requested front-end + textual-IR milestone:

- LLVM framework libraries, downloading/building LLVM, machine-code

generation, target ABI/data layout, linking, and program execution.

- Runtime implementations for helper declarations such as `say`, checked

conversions, text, and list helpers.

- Tensor operations/device execution; v1 tensor values are opaque pointers.

- Module/import compilation, overload syntax, generic record declarations,

explicit generic call-site arguments, and syntax not accepted by grammar v1.

- Full lifetime/escape analysis, LSP/editor protocol, sanitizer/fuzz/release

gates, and production compatibility guarantees.

LLVM itself is not a build dependency. If compatible `llvm-as` and `opt` executables are already installed, CMake adds optional IR validation tests; it does not download them.

Verification

The recorded validation used:

```
cmake -S . -B /tmp/indicode-build -DCMAKE_BUILD_TYPE=Release
cmake --build /tmp/indicode-build
ctest --test-dir /tmp/indicode-build --output-on-failure
```

Result: 28/28 tests passed in the environment with `llvm-as` and `opt` available. The suite covers front-end components, HIR verification, CLI output, generic/scalar IR generation, LLVM assembly and verification, and installed package consumption. Tests provide evidence for the covered cases; they are not a proof of support for language features outside the documented scope.

Related documentation

- Front-end status report — progress details.
- Front-end code audit — all 40 catalogue items.
- LLVM IR generator scope — lowering details and limits.
- Beginner guide and roadmap.
- Progress and verification details.

Appendix A: Complete 40-item Front-End Audit

Code audit of all 40 front-end entries

“Implemented and tested” means source code and focused test evidence exist at the item’s stated v1 or public-API scope. It does not claim that every API has source-language syntax or production-grade integration. The evidence below refers to implementation files and CTest targets rather than treating an earlier report as proof.

ID	PDF concept	Audit status	Code/test evidence and scope boundary
445	Source manager	Implemented/tested	`source.cpp`, `source.hpp`; `source_tests`. Owns source files and assigns IDs.
446	Source buffer	Implemented/tested	`SourceManager` owns multiple source buffers; tested for stable access and bounds.
447	Source span	Implemented/tested	`SourceLocation`/`SourceSpan` in `source.hpp`; source boundary tests.
448	Line table	Implemented/tested	Indexed CR/LF/CRLF mapping in `source.cpp`; `source_tests`.
449	Column mapping	Implemented/tested	Byte offsets, UTF-8 scalar columns, and four-column tabs; `source_tests`, `lexer_tests`, `diagnostic_tests`. Terminal display width remains unsupported.
450	UTF-8 validator	Implemented/tested	`lexer.cpp`; invalid source encodings in tokens, strings, and comments tested in `lexer_tests`.
451	Token kind	Implemented/tested	`TokenKind` and 17 keyword variants in `token.hpp`; token inventory tested in `lexer_tests`.
452	Token payload	Implemented/tested	Typed keyword, integer, float, and string payloads in `token.hpp`/`lexer.cpp`; payload/spelling/span cases in `lexer_tests`.
453	Lexer state	Implemented/tested	EOF and malformed-token recovery in `lexer.cpp`; focused and parser-robustness tests.
454	Trivia preservation	Implemented/tested	Whitespace and line/nested-block comments retain spelling and spans in `lexer.cpp`; `lexer_tests`.
455	Parser state	Implemented/tested	Explicit parser cursor and progress guard in `parser.cpp`; `parser_tests`, `parser_robustness_tests`.
456	Recursive descent parser	Implemented/tested	`parser.cpp` parses declarations, statements, generic functions, ownership modes, and effects; `grammar_tests`, `parser_tests`. Modules/import syntax and overload declarations are not parsed.
457	Pratt expression parser	Implemented/tested	`parser.cpp` precedence table; every binary operator, prefix nesting, calls/index/member tested in `grammar_tests`.
458	Grammar version	Implemented/tested	`grammar.hpp` exposes 1.0.0; grammar/spec agreement tested in `grammar_tests`. Package version 1.0.0 is a separate version identifier.
459	AST context	Implemented/tested	`ast.hpp` ownership plus `AST_CONTEXT_V1.md`; parser tests verify AST survives parser/source destruction.
460	AST node arena	Implemented/tested	Deliberate `unique_ptr` ownership in `ast.hpp`; `ast_scalability_tests` parses/destroys 10,000 statements. No arena is used.
461	AST visitor	Implemented/tested	`ast_visitor.hpp/.cpp`; `ast_visitor_tests` covers each declaration/statement/expression kind.
462	AST printer	Implemented/tested	`ast_printer.cpp`; deterministic golden test in `ast_printer_tests`.
463	AST serializer	Implemented/tested	`ast_json.cpp` emits schema version 2; structure/schema checks in `parser_tests`, `grammar_tests`, and `cli_ast_json`.
464	Syntax recovery	Implemented/tested	Declaration, statement, and delimited-expression recovery in `parser.cpp`; `parser_tests`, `parser_robustness_tests`.
465	Error synchronization	Implemented/tested	Synchronization routines in `parser.cpp`, documented in `ERROR_RECOVERY_V1.md`; malformed-input/recovery tests.
466	Symbol identity	Implemented/tested	Stable IDs assigned in `sema.cpp` and carried in `hir.cpp`; `sema_tests`, `hir_tests`.
467	Scope table	Implemented/tested	Lexical scopes and namespaces in `sema.cpp`; `sema_tests` and independent HIR scope checks in `hir_tests`.
468	Module resolver	Implemented/tested API	`resolution.cpp` maps dotted names to `.indi` paths and diagnoses invalid/missing/ambiguous matches; `resolution_tests`. Not wired to source syntax/CLI.
469	Import resolver	Implemented/tested API	`ImportResolver` handles aliases, public exports, missing modules, and cycles in `resolution.cpp`; `resolution_tests`. Not wired to source syntax/CLI.
470	Overload candidate set	Implemented/tested API	Exact signature and named-argument candidate selection in `resolution.cpp`; `resolution_tests`. Not wired to parser/analyzer calls.
471	Name binding	Implemented/tested	`sema.cpp` resolves supported values, functions, records, fields, and types; positive/negative identity checks in `sema_tests`, `hir_tests`.

ID	PDF concept	Audit status	Code/test evidence and scope boundary
472	Type checker	Implemented/tested	`sema.cpp` plus `hir.cpp`; `sema_tests`, `hir_tests` cover expressions, calls, conversions, records, collections, tensors, and invalid inputs.
473	Constraint solver	Implemented/tested	`sema.cpp` infers generic parameters and checks `Numeric`/`Integer`; positive/negative cases in `sema_tests`.
474	Type substitution engine	Implemented/tested	`sema.cpp` substitutes inferred generic types in supported patterns/returns; `sema_tests`, `hir_tests`. No explicit call-site type arguments.
475	Constant evaluator	Implemented/tested	`const_eval.cpp`; `sema_tests` cover short-circuiting, overflow, division by zero, and the 256-node limit.
476	Control-flow validation	Implemented/tested	`sema.cpp` checks conditions, counts, reachability and return typing; `sema_tests`, `hir_tests`. Loops are conservatively not assumed to execute.
477	Definite return analysis	Implemented/tested	`sema.cpp` and independent `hir.cpp` validation; nested branches, unreachable statements, and loop paths tested.
478	Effect checker	Implemented/tested	`effects.cpp` infers `io`, `allocation`, `mutation` and transitive calls; annotation errors tested in `sema_tests`, summaries checked in `hir_tests`.
479	Ownership checker	Implemented/tested	`ownership.cpp` checks mode matching, moves, use-after-move, and call-scoped borrow conflicts; positive/negative cases in `sema_tests`, metadata in `hir_tests`. Not a lifetime/escape checker.
480	Tensor shape checker	Implemented/tested	Exact element/rank/dimension checks in `sema.cpp`/`hir.cpp`; tensor tests in `sema_tests`, `hir_tests`, and CLI example. No tensor operations/device rules.
481	Diagnostic engine	Implemented/tested	`diagnostic.cpp`; `diagnostic_tests` cover text, notes, fix-its, escaping, order, and locations.
482	JSON diagnostics	Implemented/tested	`diagnostic.cpp` emits schema version 1; golden and CLI checks in `diagnostic_tests`, `cli_json_diagnostics`, `cli_json_parser_fixit`.
483	Language server index	Implemented/tested API	`symbol_index.cpp` indexes HIR symbols, references, scopes, and revisions; `symbol_index_tests`. It is not an LSP server and requires HIR snapshots.
484	Incremental parse support	Implemented/tested	`incremental_parser.cpp` reuses unchanged top-level fragments across edits; `incremental_parser_tests` compare edits/recovery with clean parses.

Appendix B: LLVM IR Generator Details

LLVM IR Generator v0

This generator is part of the completed front-end + textual-IR milestone. See the consolidated status report and its single PDF for the combined completion and verification summary.

The generator consumes independently verified, project-owned HIR and emits textual LLVM IR. It does not download, build, or link LLVM libraries, invoke LLVM during normal compilation, produce machine code, provide a runtime, link programs, or execute them.

Completion: 100% of the documented single-file front-end v1 constructs have a textual IR lowering. This is feature-scope completion, not a claim of runtime/ABI completeness, test coverage, or production readiness. It excludes syntax and behavior outside the front-end v1 contract.

Lowered constructs

- Non-generic functions and on-demand monomorphizations of generic functions,

including nested and recursive generic calls.

- Parameters, local and top-level bindings, assignment, returns, direct calls,

and named arguments. Call arguments are evaluated in source order and emitted in parameter order.

- `Bool`, all v1 integer and floating-point types, `Char`, `Text`, records,

`List<T>`, `Range<T>`, and opaque-pointer tensor values.

- Arithmetic, comparisons, unary operators, checked-conversion helper calls,

ownership markers as value-preserving wrappers, and short-circuit Boolean logic.

- `if`/`else`/`else if`, `while`, `repeat`, and `for` over integer ranges,

lists, or text.

- Record construction and member reads; list construction and indexing; text

literals, indexing, and comparisons; range construction and iteration.

- A generated `indicode_entry` initializer when top-level statements exist.

Integer mappings follow the front end's range rules: `Byte`/`U8` map to `i8`, `Int2`/`UInt2` to `i16`, `Int`/`UInt` to `i32`, and `Long`/`ULong` to `i64`. `Bool` and `Char` map to `i1` and `i32`; floating-point types map to their corresponding LLVM types. Tensors are opaque `ptr` values: the `v1` language has no tensor operations to lower. Arithmetic is emitted without overflow flags. This mapping does not promise a platform ABI or define runtime behavior.

Explicitly outside this milestone

- Module/import compilation, overload syntax, generic record declarations,

and tensor operations are not part of front-end grammar/semantics `v1`.

- Runtime implementations for `say`, checked conversions, text, and list

helpers. The IR declares helper signatures; consumers must provide compatible implementations to link or run a program.

- Target triple, data layout, target-specific ABI, machine-code generation,

linking, and execution.

The generator reports diagnostics instead of returning partial IR for invalid or unsupported HIR. It uses opaque pointers. LLVM tools are optional and are never downloaded or required. If compatible tools already exist, CMake registers additional IR assembly/verifier tests. Manual validation is:

```
/tmp/indicode-build/indicode ir examples/ir_generics.indi > /tmp/indicode.ll
llvm-as /tmp/indicode.ll -o /tmp/indicode.bc
opt -passes=verify -disable-output /tmp/indicode.bc
```